Algorithms and Flowcharts

1. Fundamental Concepts & Easy Definitions

Algorithm

Definition: A step-by-step solution to any problem, written in pseudocode or plain English along with spoken language.

Core Characteristics:

- Sequential: Clear start-to-finish order.
- Unambiguous: Each instruction is clear.
- Finiteness: Must terminate after a finite number of steps.
- Input/Output: Takes inputs and produces defined outputs.

Flowchart

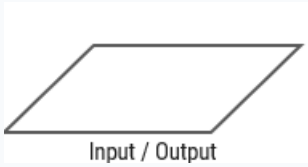
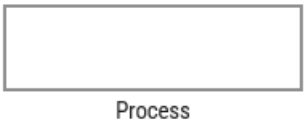
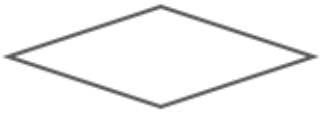
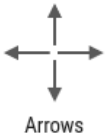
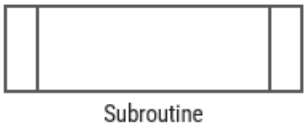
Definition: The symbolic (visual) representation of an algorithm showing the flow of the process.

Why Flowcharts Matter:

- Visual Clarity: Easier to grasp complex logic at a glance compared to long text.
- Debugging: Helps spot logical dead-ends or infinite loops easily.
- Direct Mapping: Shapes map directly to C control statements (if-else, while).

2. Standard Flowchart Symbols & C Mapping

Recognize these universal standard visual symbols and how each translates to C code:

Symbol Name	Visual Shape	Purpose & Function	Screenshot
Start / Stop	**Oval / Stadium**	Marks the entry and exit points of the algorithm program flow.	
Input / Output	**Parallelogram**	Represents reading user inputs or displaying results on screen.	
Process	**Rectangle**	Indicates calculations, memory assignments, or data processing.	
Decision Making	**Diamond**	Evaluates a relational condition (True/False or Yes/No) to branch execution.	
Lines	**Arrows (→)**	Shows the exact directional path of execution from step to step.	
Subroutine	**Predefined Rect**	Represents a predefined module, function call, or external process.	

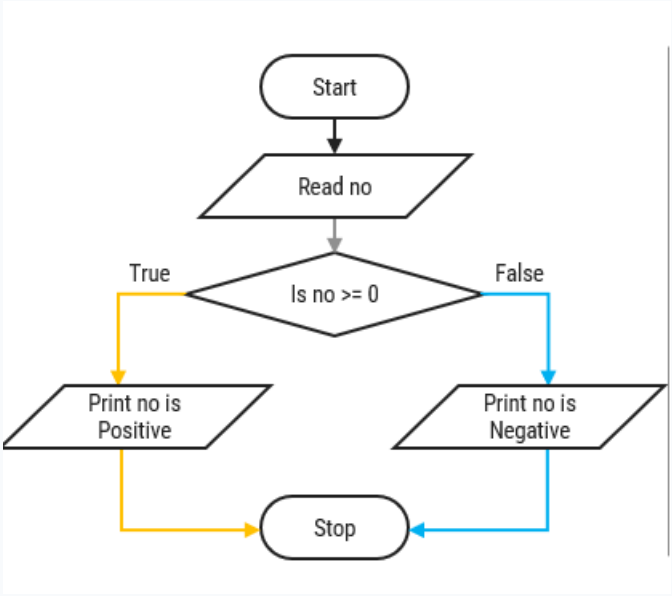
3. In-Class Examples & Blackboard Walkthroughs

Walk through these core examples step-by-step.

Example 1: Positive or Negative Number

[Algorithm]

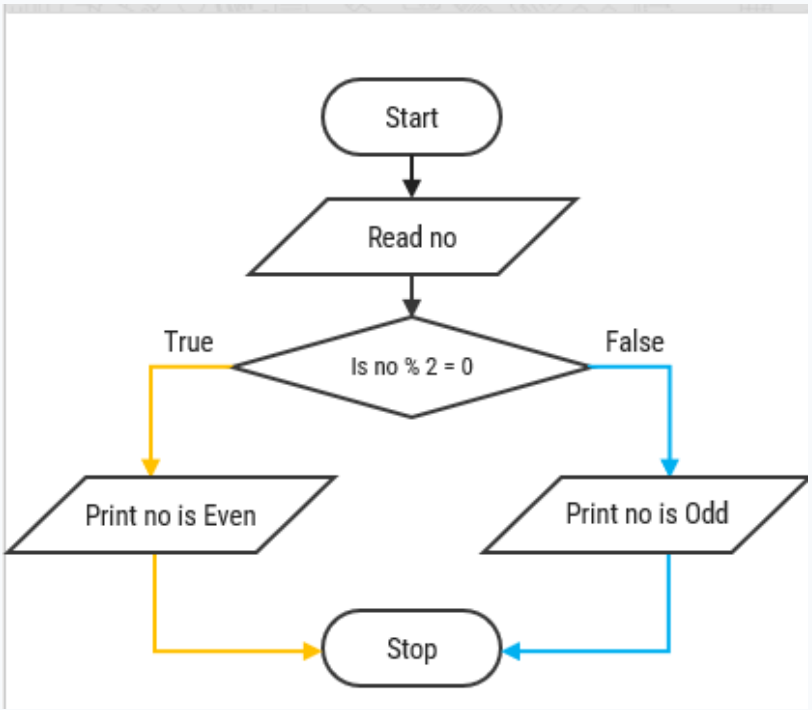
1. Step 1: Read no.
2. Step 2: If no >= 0, go to Step 4.
3. Step 3: Print no is a negative number, go to Step 5.
4. Step 4: Print no is a positive number.
5. Step 5: Stop.



Example 2: Odd or Even Number

[Algorithm]

1. Step 1: Read no.
2. Step 2: If no mod 2 = 0, go to Step 4.
3. Step 3: Print no is odd, go to Step 5.
4. Step 4: Print no is even.
5. Step 5: Stop.



Example 3: Find Largest of 2 Numbers

[Algorithm]

Step 1: Read a, b

Step 2: If a > b,

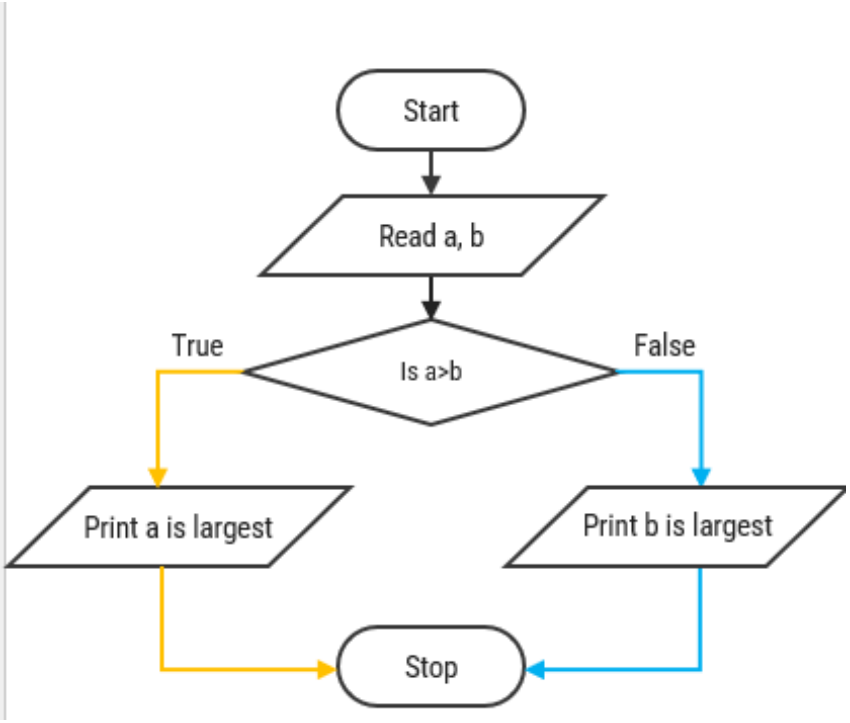
go to Step 4

Step 3: Print b is largest

go to Step 5

Step 4: Print a is largest

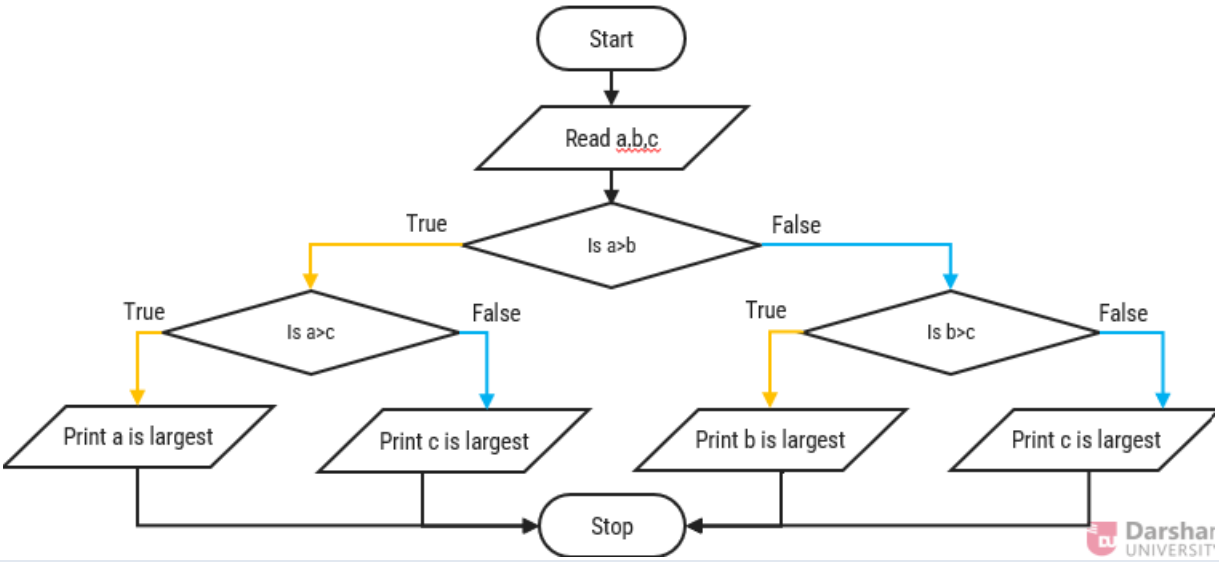
Step 5: Stop.



Example 4: Find Largest of 3 Numbers (Nested Logic)

[Algorithm]

1. Step 1: Read a, b, c.
2. Step 2: If a > b, go to Step 5.
3. Step 3: If b > c, go to Step 8.
4. Step 4: Print c is largest number, go to Step 9.
5. Step 5: If a > c, go to Step 7.
6. Step 6: Print c is largest number, go to Step 9.
7. Step 7: Print a is largest number, go to Step 9.
8. Step 8: Print b is largest number.
9. Step 9: Stop.

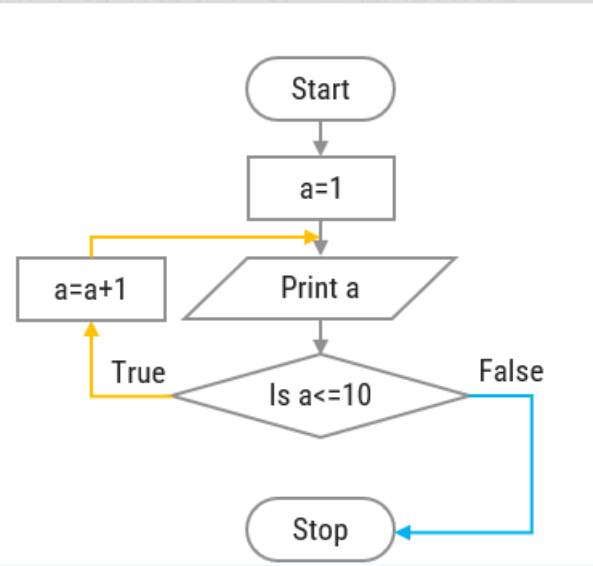


4. Advanced Control Structures: Loops, Primes & Factorials

Example 5: Looping - Print Numbers 1 to 10

[Algorithm]

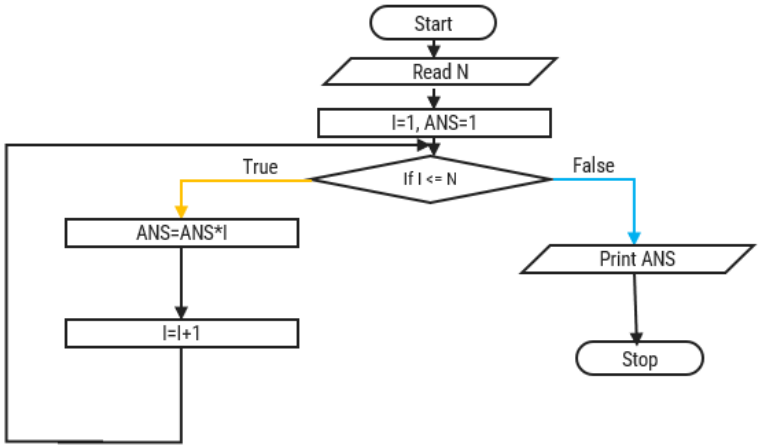
Step 1: Initialize a = 1 | Step 2: Print a | Step 3: Repeat step 2 until a <= 10 (Step 3.1: a = a + 1) | Step 4: Stop.



Example 6: Prime Number Check

[Flowchart Execution Steps]

1. Start Read N.
2. Initialize I = 2, count = 0.
3. Check Loop: If I <= N - 1:
   - If N % I == 0: count = count + 1.
   - Increment I = I + 1. Loop back.
4. Final Evaluation: If count == 0:
   - True: Print 'Prime'.
   - False: Print 'Not Prime'.
5. Stop.



Example 7: Factorial Calculation

[Flowchart Execution Steps]

1. Start Read N.
2. Initialize I = 1, ANS = 1.
3. Condition Check: If I <= N:
   - True: Calculate ANS = ANS * I.
   - Increment I = I + 1.
   - Loop back to condition check.
4. False: Print ANS.
5. Stop.

6. Interactive In-Class Exercises & TA Prompts

Spot-The-Bug Challenge (Board Activity)

Draw a broken loop flowchart on the board where a = a + 1 is omitted in the counter loop, or where a decision diamond has no True/False labels. Ask students:

"What happens when a computer tries to execute this flowchart?"

Goal: Teach students that missing loop increments lead to infinite loops in C (while(1) freeze).

Discussion Question for Students

Question: "Why do we start checking prime numbers from I = 2 up to N - 1 instead of starting from I = 1?"

Expected Answer: Every integer is divisible by 1, so starting from 1 would always increment count and make every number appear non-prime!

Summary Checklist

Checklist Items

- Ensure all students know the 5 basic flowchart shapes and their C syntax equivalents.
- Remind students that decision diamonds MUST have two outgoing branches labeled True/False or Yes/No.
- Verify that students always draw explicit arrowheads to indicate execution flow.
- Connect algorithms directly to upcoming C concepts: if-else (Decision), for/while (Looping), and scanf/printf (I/O).